

Krafton Jungle GameTech-Lab

WEEK 06+

— Forward Rendering 기반 Multi-pass 효과 렌더링

2026/04/16

8팀: 김연하 정찬일 박세영

1. 학습 목표 및 주요 키워드

"Forward 렌더링 결과물을 기반으로 여러 Post-Processing 효과 구현하기"

- Projection Decal
- Post-Processing
- Multipass Rendering
- Forward Rendering
- Render State
- Intersect AABB / OBB

2. 비용 분석 관점

"Geometry Cost"

: Vertex Shader 및 triangle 처리 비용

"Pixel Cost"

- Per-pixel Compute : Pixel Shader 연산 비용
- Overdraw / Fillrate : 동일 픽셀 중복 처리 비용

"Bandwidth Cost"

: 텍스처 및 렌더 타겟 읽기·쓰기 비용

"Scalability"

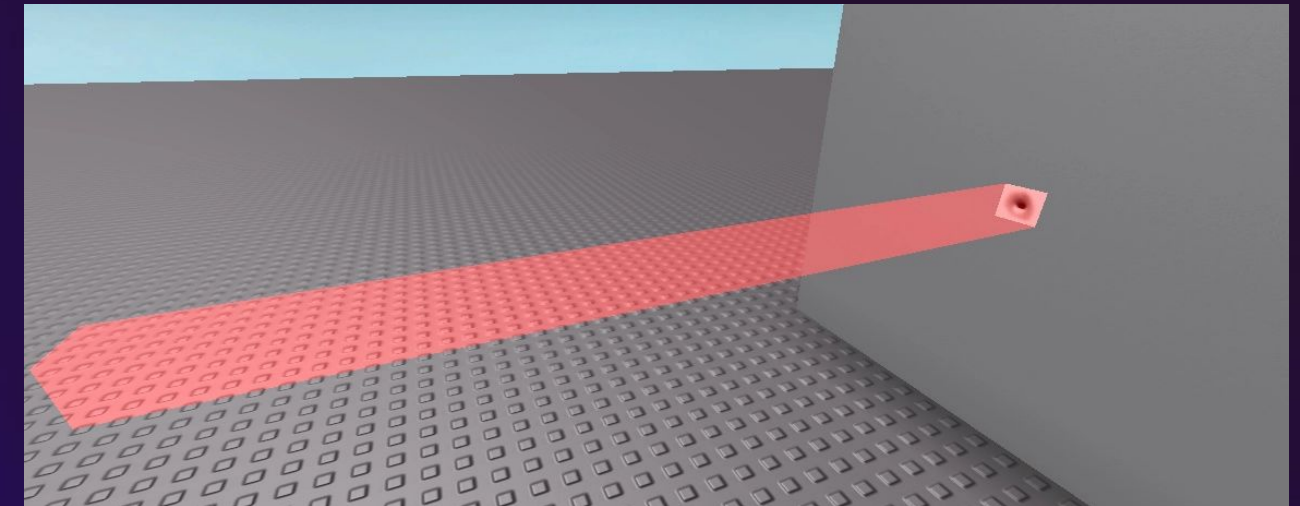
: 후보 수 증가에 따른 비용 증가 양상

Projection Decal

- 기존 장면 결과를 기반으로 합성되는 효과
- 기존 Pass에 포함시, 모든 Shader에 Decal 로직이 결합됨
→ 복잡도와 계산 비용 증가



기존 Pass 결과를 기반으로
별도 Pass에서 적용 및 합성



“Scene의 Geometry에 Volume을
투영”

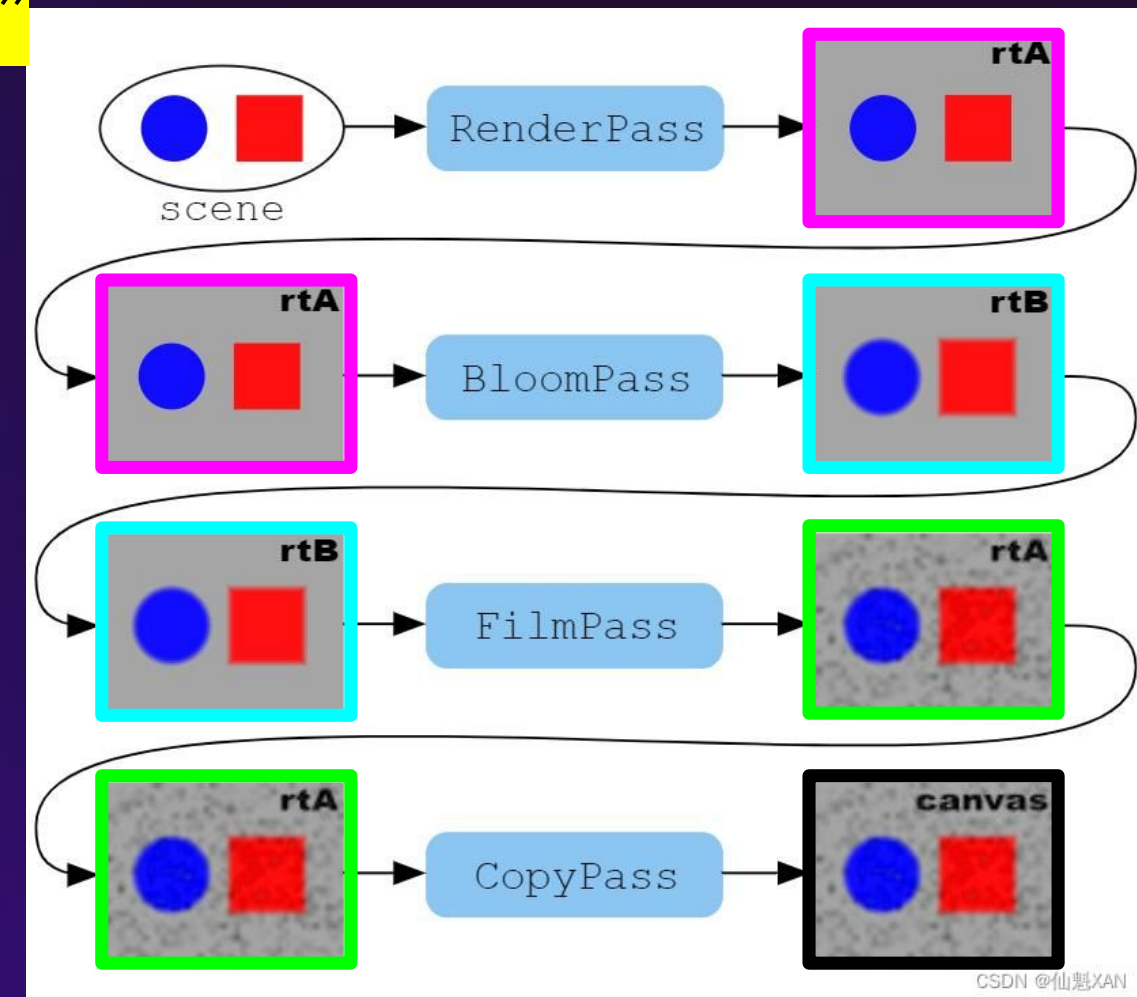
Multi-Pass Rendering

렌더링을 여러 Pass로 나누어
순차적으로 처리

각 Pass의 결과를 텍스처로
저장하고, 다음 Pass에서 입력으로
사용

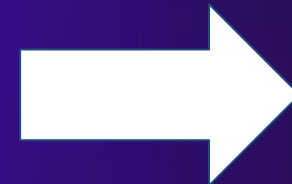
“텍스처는 Pass 간 데이터를 전달하는

매개체”

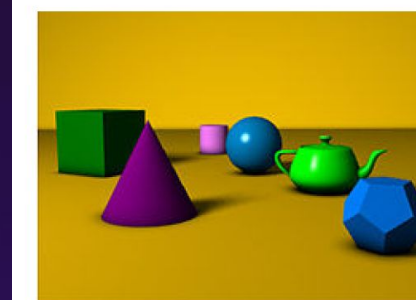


Forward Rendering

Decal을 적용하려면
기존 렌더링 결과를 활용해야 함



Forward Rendering이
어떤 정보를 남기는지가 관건



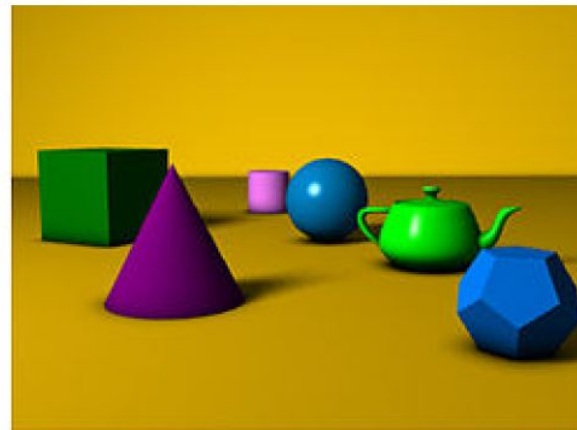
Scene Color



Scene Depth

Forward Rendering

“화면 정보를 한데 모아 저장해두기”



Scene Color



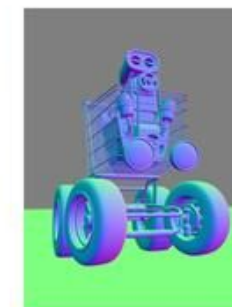
Scene Depth

VS

“화면 정보를 나눠서 저장해두기”



Main



Normals



Material ID



Object ID



Alpha



Depth



Roughness



Fresnel



Ambient Occlusion

Forward Rendering

화면 정보를
“한데 모아” 저장해두기

→ Decal 적용시, 월드 좌표 복원 필요
→ GPU의 픽셀 계산 비용 ↑

→ Compute-Bound 성향

VS

화면 정보를
“나눠서” 저장해두기

→ GPU의 각 텍스처에 대한
Read · Write 비용 ↑

→ Bandwidth-Bound 성향

Forward Rendering

화면 정보를
“한데 모아” 저장해두기

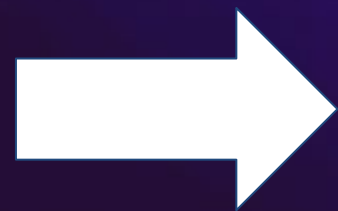
Decal 적용시, 월드 좌표 복원 필요
→ GPU의 픽셀 계산 비용 ↑

Compute-Bound 성향

“Forward 렌더링에서
데칼 적용 시,
Compute-Bound
성향이 강해짐”

Forward Post-process 최적화 방향

Post-processing(후처리)는 대부분 픽셀 기반
→ 전체 Per-pixel Compute 비용 절감이 관건



후보 축소 (Broad Phase)
: 계산 대상 수 줄이기

Per-pixel 연산 최소화
: 계산 단가 줄이기

Per-pixel 연산 비용 최소화

월드 좌표 복원 연산 최소화

- Vertex Shader 및 triangle 처리 비용
- 가능한 계산은 CPU에서 미리 처리

Fullscreen Triangle 적용

- Quad 대신 Triangle 1개로 Screen-space pass 처리
- 불필요한 vertex 및 경계 오버헤드 감소

클러스터 기반 데칼 볼륨 분배 최적화

- View-space Depth 기반
- 비선형 Z-slice 분할로 깊이별 분포 최적화

Projection Decal : 접근법 비교

방법1. Mesh-based

- Clipping Geometry 생성
- Buffer 업로드 필요
- 데칼 변경 시 geometry 재생성 필요

방법2. Screen-space

- 월드 위치 복원 후 볼륨 내부 여부 판정
 - Per-pixel 계산 비용 증가
- geometry 재생성 없이 동적 변경 대응

Projection Decal : 접근법 비교

"Geometry Cost"

: Mesh-based에서 크게 증가

"Pixel Cost"

: Screen-space에서 크게 증가

"Bandwidth Cost"

- Mesh-based: GPU Buffer 업로드/갱신
- Screen-space: SceneColor, Depth 버퍼 읽기

"Scalability"

: Screen-space가 후보 관리 구조로 유리



동적으로 decal 수가 늘고 바뀌는 상황에서
유리한

“Screen-space” 방식 선택

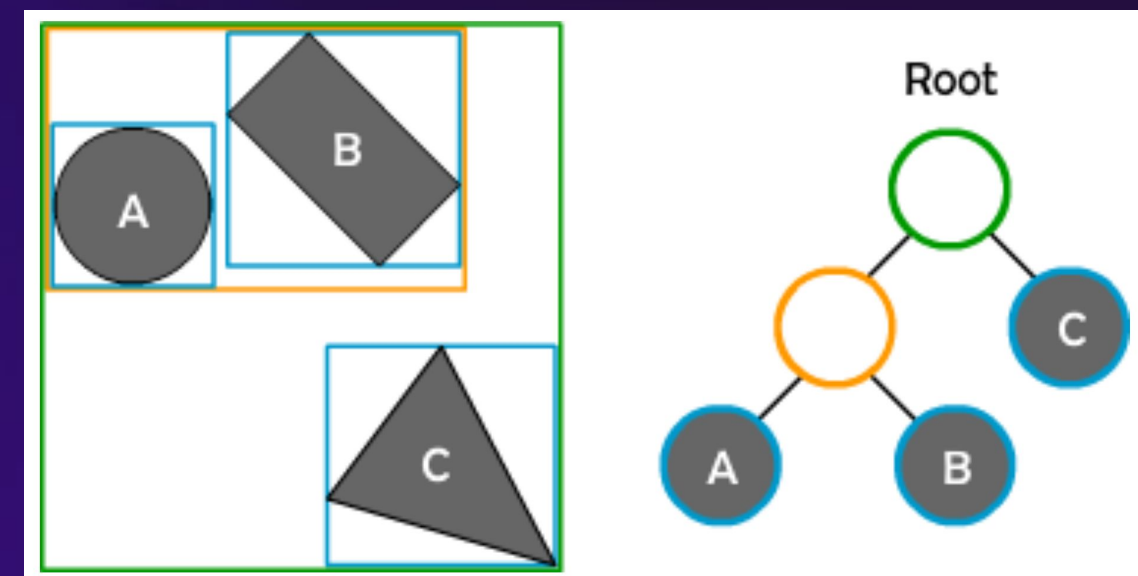
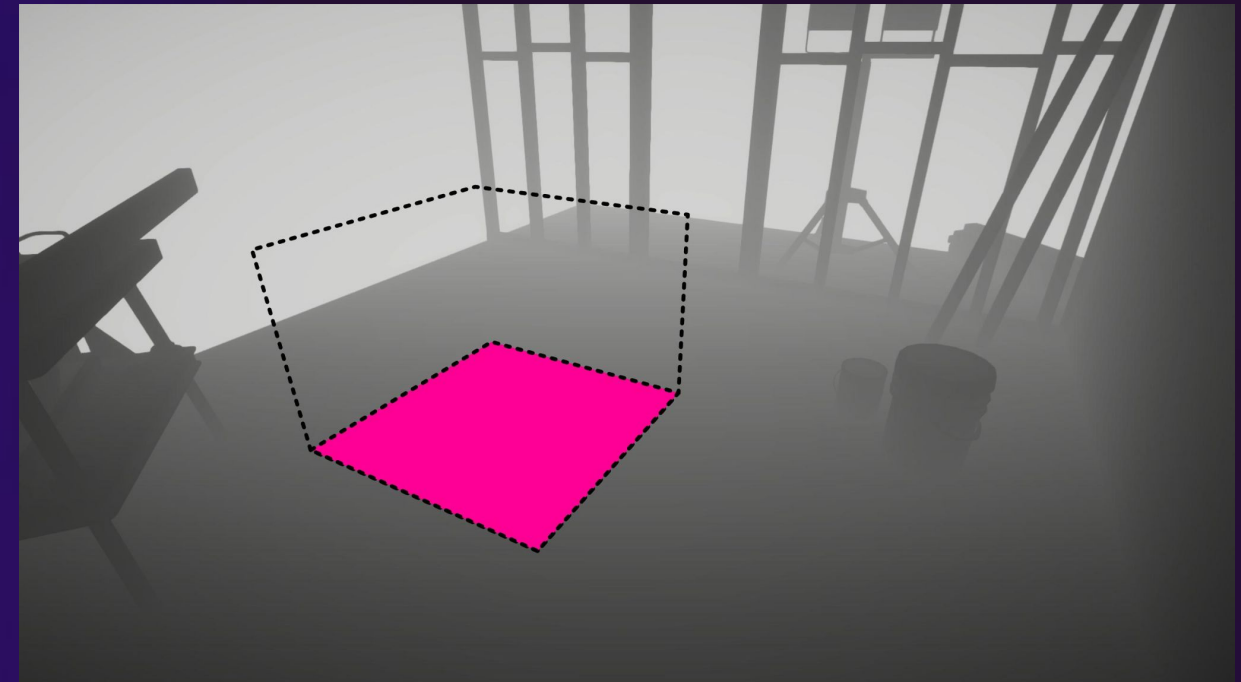
전략1. Volume Draw

- 데칼마다 자신의 볼륨(박스)을 직접 그림
- 그 볼륨 안에 들어오는 픽셀에 데칼 적용
 - Depth 버퍼를 이용해 월드 좌표 복원
 - 복원된 좌표와 볼륨의 Inside-Test 수행

후보 축소 (Broad Phase)

: Decal Volume-Object 교차 검사 수행

- AABB-OBB 교차 검사
- BVH 트리 구조 사용



문제: Per-pixel 비용 폭발

기존 방식

- 모든 픽셀이 모든 데칼을 검사
- 데칼 수가 늘어날수록 픽셀당 검사 횟수 증가
 - 총 비용 = 픽셀 수 × 데칼 수

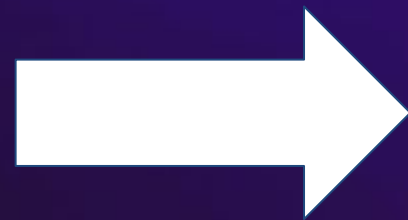


"각 픽셀이 필요한 데칼만 보게 할 수 있을까?"

전략 2. Clustered-Lookup

"모든 데칼을 보지 말고, 내 주변에 있는 데칼만 보자"

1. CPU가 사전에 각 클러스터에 해당하는 데칼을 등록해둔다
 2. 픽셀은 해당하는 Cluster의 데칼만 조회한다
- > 전체 검사 대신 후보만 검사하여 per-pixel 비용 감소

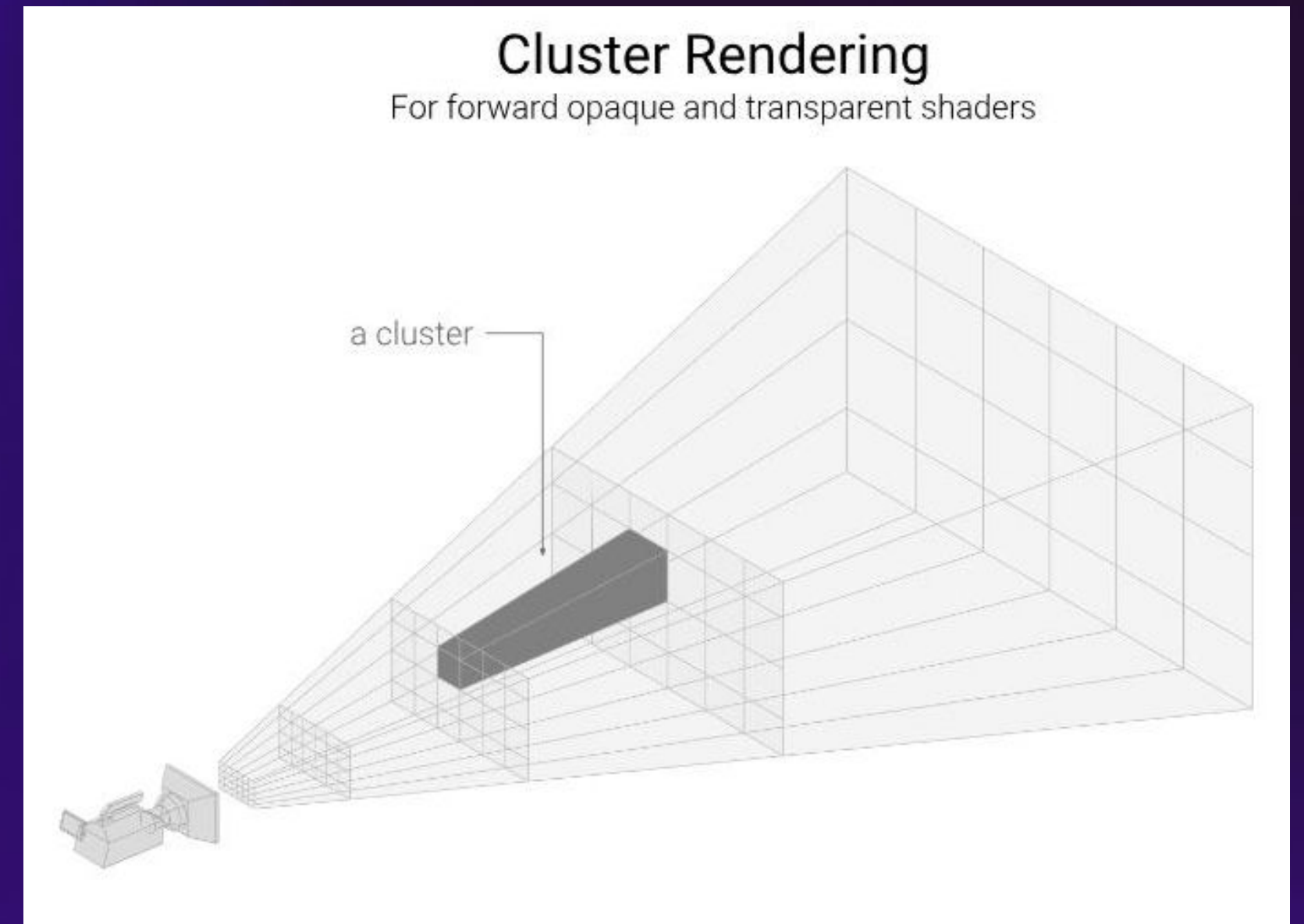


총 비용 = 픽셀 수 × 평균 후보 수

전략 2. Clustered-Lookup

- 각 클러스터는 화면 상의 작은 공간 + 깊이 범위를 의미

1. 화면을 3D 클러스터로 분할
2. 데칼 볼륨을 클러스터에 등록 / 갱신 (CPU)
3. 각 픽셀이 클러스터 조회 후 데칼 적용
(GPU, Per-pixel)



Screen-space Decal : 전략 비교

전략1. Volume Draw

- 후보 축소 시점: Draw 이전
- 후보 축소 방식: Scene BVH
- 처리 단위: Decal volume별 Draw Call

- Bandwidth ↓

- Overdraw 있음
- Scalability ↓

전략2. Clustered-Lookup

- 후보 축소 시점: Pixel Shader 내부
- 후보 축소 방식: 스크린 공간 Cluster 조회
- 처리 단위: Pixel별 Lookup

- Overdraw 없음
- Scalability ↑

- Bandwidth ↑
- Lookup 비용 발생

“Volume Draw” vs “Clustered Lookup”

항목	Volume Draw	Clustered (정적, 캐싱됨)	Clustered (동적 갱신)
Build Time	0.007 ms	0.000 ms	0.192 ms
Shading	0.235 ms	0.004 ms	0.004 ms
Draw Calls	2670	1 (fullScreen)	1 (fullScreen)
GPU Upload	없음	~522 KB	~525 KB

Shading 비용

- Volume Draw: 0.235 ms
- Clustered: 0.004 ms

=> 약 60배 감소

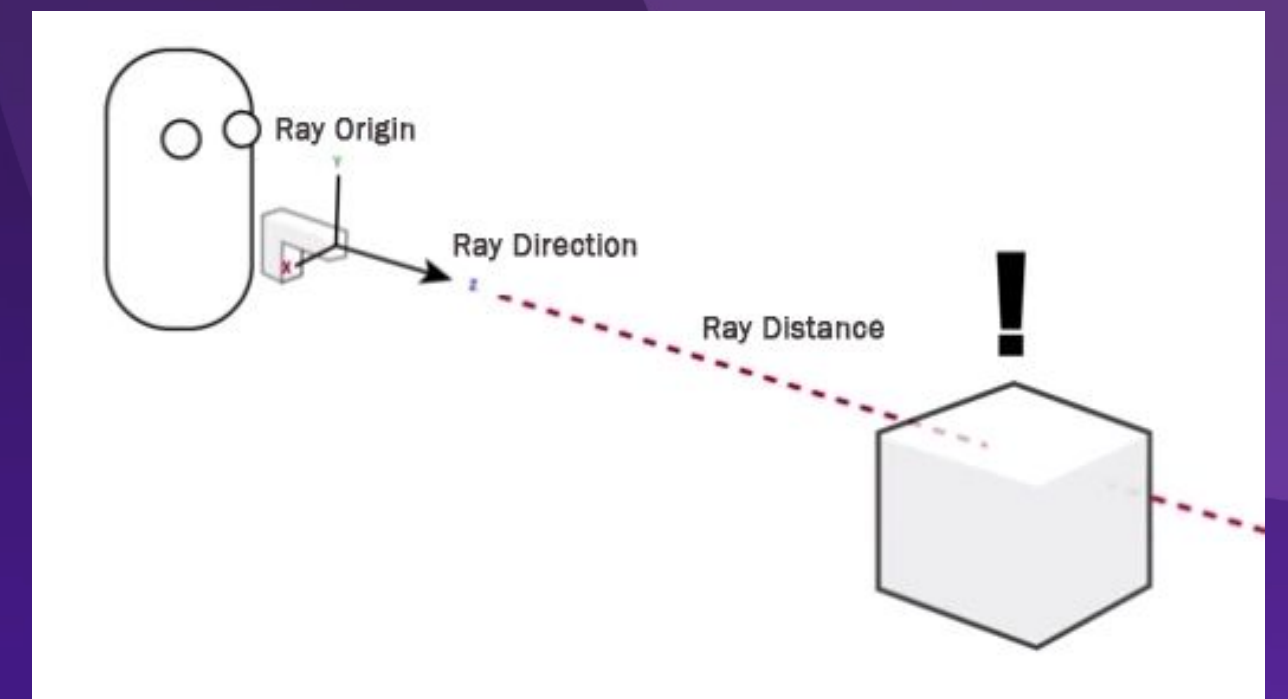
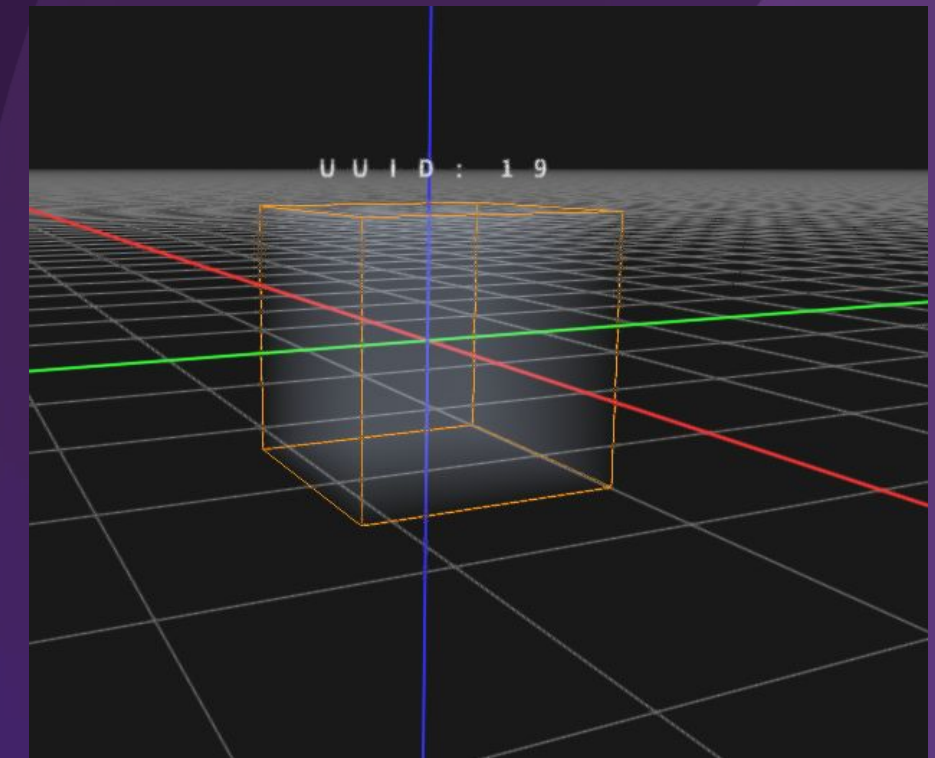
Draw Call

- Volume Draw: 2670
- Clustered: 1 (fullscreen)

=> 2670배 차이

4. 확장 구현: Local Height Fog

- Global / Local Height Fog를 구분
- Decal Volume과 동일한 cluster 구조를 적용
- Ray-Volume 교차 구간 기반 적분 수행
 - 간소화된 적분: 샘플 값 2개 (25%, 75%)만을 이용



시연

